Projekt numeriska metoder (SF1546)

Värmeöverföring: Från teori till numerisk lösning

Författare: Sebastian Alin, Robert Falk, Pontus Graf von Hasslingen

Datum: 2026-04-23

Innehållsförteckning

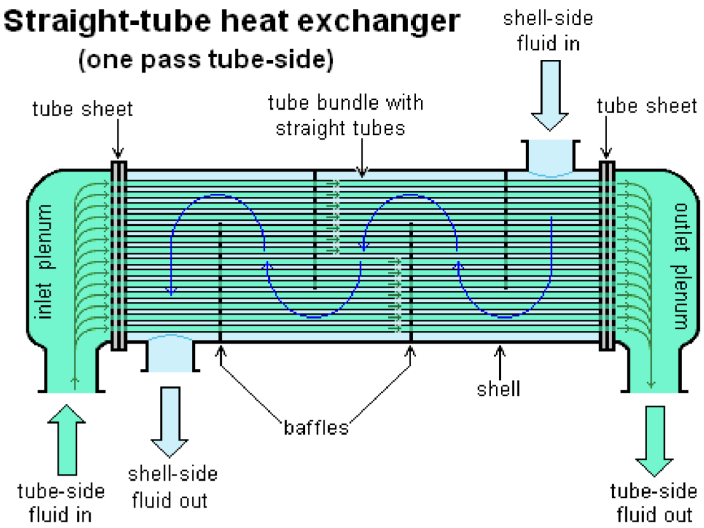
1 Introduktion . . . 3
2 Metod . . . 4
2.1 Del 1.1 . . . 4
2.2 Del 1.2 . . . 4
2.3 Del 2.1 . . . 5
2.4 Del 2.2 . . . 5
3 Resultat . . . 6
3.1 Del 1.1 . . . 6
3.2 Del 1.2 . . . 8
3.3 Del 2.1 . . . 10
3.4 Del 2.2 . . . 11
4 Diskussion . . . 11
5 Slutsats . . . 11

1 Introduktion

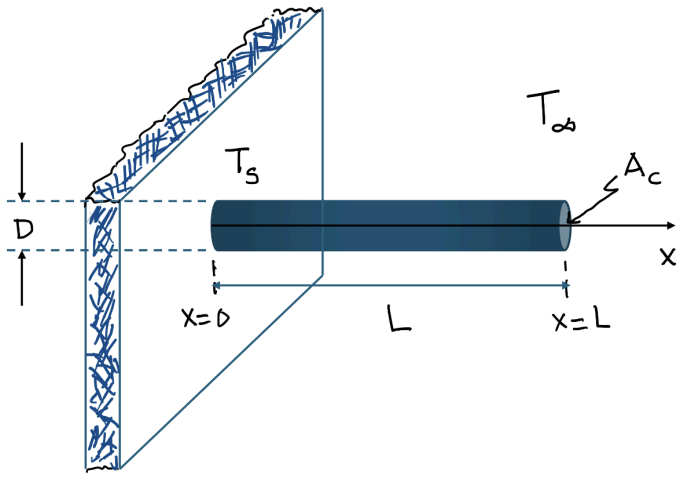
Detta prjekt är en del av kursen SF1546 - Projekt numeriska metoder, och syftar till att tillämpa numeriska metoder för att lösa problem inom värmeöverföring i en värmeväxlare och kylflänsar.

Målet med projektet är att lösaproblem kopplade till värkligheten, med numeriska metoder, särskill de som inehåller ickelinjära ekvationssystem och randvärdesproblem. Genom att använda dessa metoder, kommer vi att kunna analysera och optimera värmeöverföringen i de givna systemen.

Projektet är indelat i två delar, där den första delen syftar till att lösa ett icke-linjärt ekvationssystem som beskriver värmeöverföringen i en värmeväxlare, med hjälp av Newtons metod ($x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}$), för att få beräkna den maximala ytterdimatern $\boldsymbol{d}$ och sedan innderdiameter D_s.



Den andra delen fokuserar på att lösa ett randvärdesproblem som beskriver värmeöverföringen i en kylfläns. Med hjälp av diskretisering, med centrerad differns, och finita differnsmetoden, beräknas temperaturfördelningen längs kylflänsen för material med olika egenskaper.



2 Metod

2.1 Del 1.1

Det första problemet kan beskrivas av följande icke-linjära ekvationssystem med givna parametrar (Tabell 1):

$$Q(d) = A_{ex} \cdot U_f \cdot \Delta T_m \tag{1}$$

$$U_f = \frac{1}{\frac{d}{D_s \cdot h_t} + d \cdot \frac{R_{fi}}{D_s} + \frac{d \cdot \ln(d/D_s)}{2k_w} + R_{fo} + \frac{1}{h_s}} \tag{2}$$

Parameter	**Värde**	**Enhet**
R_{fi}	$1.76 \cdot 10^{-4}$	$m^2{}^\circ C/W$
R_{fo}	$1.76 \cdot 10^{-4}$	$m^2{}^\circ C/W$
h_s	356	$W/(m^2{}^\circ C)$
h_t	356	$W/(m^2{}^\circ C)$
k_w	60	$W/(m^\circ C)$

Tabell 1: parametrar för värmeöverföringen i värmeväxlaren

Specefikationer för värmeöverföringen i värmeväxlaren är som följer: $\Delta T_m = 29.6^\circ C$ och $Q = 801368W$; den minsta ytarean som upfyller specefkationerna är given till $A_{ex} = 64.15m^2$ och skalets innerdiameter är $D_s = 1.219m$.

Alla parametrar förutom d är kända, och genom att konstruera en funktion $f(d)$ som representerar skillnaden mellan den beräknade värmeöverföringen och den givna värmeöverföringen, kan problemet formuleras som att hitta roten till denna funktion; det vill säga där $f(d) = 0$.

$$f(d) = A_{ex} \cdot U_{f(d)} \cdot \Delta T_m - Q = 0 \tag{3}$$

För att lösa denna rot, användes Newtons metod, som iterativt närmar sig lösningen genom att uppdatera gissningen baserat på funktionen och dess derivata.

$$x_{d+1} = d_n - \frac{f(d_n)}{f'}(d_n) \tag{4}$$

2.2 Del 1.2

Det andra problemet bygger på Avsnitt 2.1 (del 1.2) med två ytterligare ekvationer och ginva parametrar (Tabell 2), där både d och D_s är okända:

$$\Delta P = \frac{c}{D_s^2 (S_t - d)^2} \tag{5}$$

$$A_{ex} = \pi K_1 \frac{D_1^{n_1}}{d^{n_1 - 1}} \tag{6}$$

Parameter	Värde	Enhet
c	0.389	$Ws \cdot m$
S_t	0.016	m
K_1	0.249	m
n_1	2.207	

Tabell 2: Givna parametrar för värmeväxlaren.

Specefikationerna för värmeöverföring är samma som i del 1.1 ($\Delta T_m = 29.6°C$ och $Q = 801368W$), och en given maximal tryckskillnad över värmeväxlaren $\Delta P = 49080Pa$.

och genom att använda de två nya ekvationerna, kan både d och D_s lösas samtidigt genom att konstruera ett system av två icke-linjära ekvationer och använda newtons metod för system.

Ekvation 1 kan nu omformuleras till att inkludera D_s som en variabel, där U_f och A_{ex} är funktioner av både d och D_s:

$$Q(d, D_s) = A_{ex}(d, D_s) \cdot U_f(d, D_s) \cdot \Delta T_m \tag{7}$$

$$\Rightarrow F_1(d, D_s) = A_{ex}(d, D_s) \cdot U_f(d, D_s) \cdot \Delta T_m - Q = 0 \tag{8}$$

Ekvation 5 ger en ytterligare ekvation som relaterar d och D_s:

$$F_2(d, D_s) = \frac{c}{D_s^2 (S_t - d)^2} - \Delta P = 0 \tag{9}$$

Dess funktioner kan tillsamans konstrueras som en vektorvärd funktion, $\vec{F} = (F_1, F_2) = \vec{0}$. Detta system av icke-linjära ekvationer kan sedan lösas med hjälp av Newtons metod för system, där $\vec{x} = (d, D_s)$:

$$\vec{x}_{n+1} = \vec{x}_n - J^{-1}\vec{F}(x_n) \tag{10}$$

2.3 Del 2.1

Denna del går ut på att teoretiskt härleda approximationen i Ekvation 12 med hjälp av Taylorytveckling, och implementera tre olika approximationer i *Python* för att derivatan av $\mathbf{sin(e^x)}$ vid $x = 0.75$.

$$f(x_j) = \frac{-\omega_{j+2} + 4\omega_{j+1} - 3\omega_j}{2h} + \mathcal{O}(h^2) \tag{11}$$

$$f'(x_j) = \frac{3\omega_j - 4\omega_{j-1} + \omega_{j-2}}{2h} + \mathcal{O}(h^2) \tag{12}$$

$$f'(x_j) = \frac{\omega_{j+1} - (\omega_{j-1})}{2h} + \mathcal{O}(h^2) \tag{13}$$

$\omega_j = f(x_j) = f(a + jh)$ där a är startpunkten, b är slutpunkten, och h är steglängden $h = \frac{b-a}{N}$, där N är antalet delintervall.

2.4 Del 2.2

Denna del fokuserade på att lösa ett randvärdesproblem som beskriver av differensialekvationen

$$\frac{d^2T}{dx^2} = \alpha_1(T - T_\infty) + \alpha_2(T^4 - T_\infty^4), \qquad x \in [0, L] \tag{14}$$

Samt dirichlet randvillkor:

$$T(0) = T_s, \qquad T(L) = T_{\text{inf}} \tag{15}$$

Detta problem kan lösas genom att diskritetisera andra ordningens derivata med central differens, där den andra derivatan approximativt kan skrivas som:

$$\frac{d^2T}{dx^2} \approx \frac{T_{j+1} - 2T_j + T_{j-1}}{h^2} \qquad (+\mathcal{O}(h^2)) \tag{16}$$

Där $h = \frac{L}{N}$, där N är antalet delintervall.

Vidare kan uttrycket skrivas för varje givet $T_j = T(hj)$ som en funktion: $F_j\big(T_{j-1}, T_j, T_{j+1}\big) = 0$, där $T_0 = T_s$ och $T_N = T_\infty$. Detta ger upphov till ett system av icke-linjära ekvationer ($\vec{F} = (F_1, F_2, ..., F_{N-1}) = \vec{0}$) som kan lösas med hjälp av Newtons metod för system, $\vec{T}^{k+1} = \vec{T}^k - J^{-1}\vec{F}\big(T^k\big)$.[1]

3 Resultat

3.1 Del 1.1

3.1.1 Uppgift 1

Denna del uppgift skall formulera den mattematiska modellen för att implementera newtons metod, genom att beräkna $f(d)$ och dess derivata $f'(d)$ analytiskt. $f'(d)$ kan beräknas genom att använda kedjeregeln och produktregeln för derivering av vilket ger:

$$f'(d) = -A_{ex} \cdot \Delta T_m \cdot U_f{}'(d) \tag{17}$$

$U_f{}'(d)$ kan beräknas genom att använda kvotregeln för derivering, där $g(d)$ är nämnaren i uttrycket för $U_{f(d)}$:

$$g(d) = \frac{d}{D_s \cdot h_t} + d \cdot \frac{R_{fi}}{D_s} + \frac{d \cdot \ln(d/D_s)}{2k_w} + R_{fo} + \frac{1}{h_s} \tag{18}$$

$$U_{f(d)} = \frac{1}{g(d)} = g(d)^{-1} \Rightarrow U_f{}'(d) = -\frac{1}{g'(d)^2} \tag{19}$$

$g(d)$ är relativt enkelt att derivera förutom den logaritmiska termen, där och produktregeln måste användas.

$$\frac{d}{dd}(d \cdot \ln(d/D_s)) = 1 \cdot \ln(d/D_s) + d \cdot \frac{1}{d} = \ln(d/D_s) + 1 \tag{20}$$

[2] Detta ger derivatan:

$$g'(d) = \frac{1}{D_s \cdot h_t} + \frac{R_{fi}}{D_s} + \frac{\ln(d/D_s) + 1}{2k_w} + \left(\frac{1}{2k_w}\right) \tag{21}$$

[1] Notera $\overrightarrow{T^k}$ används som superskript och inte som exponent

[2] $\frac{d}{dd}(\ln(d/D_s)) = \frac{d}{dd}(\ln(d) - \ln(D_s)) = \frac{1}{d} - 0$

$$\Rightarrow f'(d) = -\frac{A_{ex} \cdot \Delta T_m}{\left(\frac{1}{D_s \cdot h_t} + \frac{R_{fi}}{D_s} + \frac{\ln(d/D_s)+1}{2k_w}\right)^2} \tag{22}$$

3.1.2 Uppgift 2

Denna uppgift skall implementera Newtons metod i *Python*, med den beräknade derivatan, för att iterativt närma sig lösningen för d, med en startgissning $d_0 = 0.007m$ och en tolerens på 10^{-8} (Givet från uppgiften).

```python
def newton_method(d_0, tolerance, max_iter=10000):
    d_n = d_0
    errors = []
    for i in range(max_iter): # Loop until max iterations

        if f_prime(d_n) == 0:  # Avoid division by zero
            print("Derivative is zero. No solution found.")
            return None

        d_next = d_n - f(d_n) / f_prime(d_n) # newtons method formula
        errors.append(abs(d_next - d_n)) # store the error for convergence analysis

        if abs(d_next - d_n) < tolerance:
            print(f"Converged to {d_next} after {i+1} iterations.")
            return d_next, errors

        d_n = d_next

    print("Maximum iterations reached. No solution found.")
    return None
```

Denna lösning ger svaret $\boldsymbol{d = 0.00746192m}$ efter endast 3 iterationer, vilket visar på en snabb konvergens.

3.1.3 Upgift 3

Denna uppgift skall analysera konvergensordningen för Newtons metod genom att använda de beräknade felen från varje iteration. och jämföra lösningen mod pythons egna lösning

a) genom att anta felet som en potensfunktion kan konvergensordningen p uppskattas genom att använda följande formel:

$$e_{k+1} \approx Ce_k^p \Rightarrow e_{k+2} \approx Ce_{k+1}^p \tag{23}$$

$$\frac{e_{k+2}}{e_{k+1}} = \frac{Ce_{k+1}^p}{Ce_k^p} = \left(\frac{e_{k+1}}{e_k}\right)^p \Rightarrow \ln\left(\frac{e_{k+2}}{e_{k+1}}\right) = p\ln\left(\frac{e_{k+1}}{e_k}\right) \Rightarrow p = \frac{\ln(e_{k+2}/e_{k+1})}{\ln(e_{k+1}/e_k)} \tag{24}$$

Implementeringen av detta i *Python* ger en uppskattning av konvergensordningen $\boldsymbol{p = 1.9901146 \approx 2}$, och stämmer väl överens med den teoretiska konvergensordningen för Newtons metod, som är 2.

b) Lösningen från Scipy: (`optimized_solution = optimize.fsolve(f, d_0)`) ger $\boldsymbol{d = 0.00746195m}$, vilket är i praktiken identiskt med vår implementerade lösning, och bekräftar att vår implementation av Newtons metod är korrekt och effektiv.

c)

3.2 Del 1.2

3.2.1 Uppgift 4

Denna uppgift skall formulera det icke-linjära ekvationssystemet så att det kan lösas med Newtons metod för system. I metoden konstruerades en vektorvärd funktion $\vec{F} = (F_1, F_2)$ där F_1 (Ekvation 8) och F_2 (Ekvation 9) är de två ekvationerna som beskriver systemet. För att implementera Newtons metod för system ($\vec{x}_{n+1} = \vec{x}_n - J^{-1}\vec{F}(x_n)$), krävs även Jacobimatrisen, som består av de partiella derivatorna av F_1 och F_2 med avseende på variablerna d och D_s. Jacobimatrisen J kan skrivas som:

$$J = \begin{bmatrix} \frac{\partial F_1}{\partial d} & \frac{\partial F_1}{\partial D_s} \\ \frac{\partial F_2}{\partial d} & \frac{\partial F_2}{\partial D_s} \end{bmatrix} \tag{25}$$

$$F_2 = \frac{c}{D_s^2(S_t - d)^2} - \Delta P = \frac{c}{h(d, D_s)} - \Delta P \qquad \left(h(d, D_s) = D_s^2(S_t - d)^2\right) \tag{26}$$

$$\Rightarrow \frac{\partial F_2}{\partial d} = -\frac{c}{h(d, D_s)^2}\frac{\partial h}{\partial d} = \frac{c(2D_s^2(S_t - d))}{D_s^4(S_t - d)^4} = \frac{2c}{D_s^2(S_t - d)^3} \tag{27}$$

$$\Rightarrow \frac{\partial F_2}{\partial D_s} = -\frac{c}{h(d, D_s)^2}\frac{\partial h}{\partial D_s} = \frac{c\left(2D_s(S_t - d)^2\right)}{D_s^4(S_t - d)^4} = \frac{2c}{D_s^3(S_t - d)^2} \tag{28}$$

$$F_1 = A_{ex}\frac{d, D_s}{g(d, D_s)} \cdot \Delta T_m - Q, \qquad \left(g(d, D_s) = 1/U_f\right) \tag{29}$$

$$\Rightarrow \frac{\partial F_1}{\partial d} = \Delta T_m \frac{\frac{\partial A_{ex}}{\partial d} \cdot g(d, D_s) - A_{ex} \cdot \frac{\partial g}{\partial d}}{g(d, D_s)^2} \tag{30}$$

$$\Rightarrow \frac{\partial F_1}{\partial D_s} = \Delta T_m \frac{\frac{\partial A_{ex}}{\partial D_s} \cdot g(d, D_s) - A_{ex} \cdot \frac{\partial g}{\partial D_s}}{g(d, D_s)^2} \tag{31}$$

$$A_{ex} = \pi K_1 D_1^{n_1} \cdot (d^{1-n_1}) \Rightarrow \quad \frac{\partial A_{ex}}{\partial d} = -\pi K_1 D_1^{n_1}\frac{n_1 - 1}{d_1^n} \qquad \frac{\partial A_{ex}}{\partial D_s} = \pi K_1 n_1 \frac{D_1^{n_1 - 1}}{d^{n_1 - 1}} \tag{32}$$

$\frac{\partial g}{\partial d}$ har redan beräknats i Ekvation 21

$$\frac{\partial g}{\partial d} = \frac{1}{D_s \cdot h_t} + \frac{R_{fi}}{D_s} + \frac{\ln(d/D_s) + 1}{2k_w} + \left(\frac{1}{2k_w}\right) \tag{33}$$

$$\frac{\partial g}{\partial D_s} = -\frac{d}{D_s^2 \cdot h_t} - d \cdot \frac{R_{fi}}{D_s^2} - \frac{d}{2k_w D_s} \quad \left(\frac{\partial}{\partial D_s}(d \cdot \ln(d/D_s)) = \frac{1}{d/D_s} \cdot -\frac{d}{D_s^2} = -\frac{1}{D_s}\right) \tag{34}$$

Jacobimatrisen kan därmed skrivas som:

$$\begin{bmatrix} \Delta T_m \frac{\frac{\partial A_{ex}}{\partial d} \cdot g(d, D_s) - A_{ex} \cdot \frac{\partial g}{\partial d}}{g(d, D_s)^2} & = \Delta T_m \frac{\frac{\partial A_{ex}}{\partial D_s} \cdot g(d, D_s) - A_{ex} \cdot \frac{\partial g}{\partial D_s}}{g(d, D_s)^2} \\ \frac{2c}{D_s^2(S_t - d)^3} & \frac{2c}{D_s^3(S_t - d)^2} \end{bmatrix} \tag{35}$$

3.2.2 Uppgift 5

a) Denna uppgift skall implementera Newtons metod för system i *Python*, med den beräknade Jacobimatrisen, för att iterativt närma sig lösningen för d och D_s, med startgissningen $d_0 = 0.015m$ och $D_{s0} = 0.8m$, och en tolerens på 10^{-8}; samt analysera konvergensen av lösningen.

```
def jacobian(d, D):
    return np.array([[dF1dd(d, D), dFdD(d, D)],
                     [dF2dd(d, D), dF2dD(d, D)]])

# implement the Newton's method for solving the system of equations F(d, D) = 0
def newton_method_system(d_0, D_0, tolerance, max_iter=10000):
    d_n = d_0
    D_n = D_0
    errors = []
    for i in range(max_iter):
        F_n = F(d_n, D_n)
        J_n = jacobian(d_n, D_n)

        if np.linalg.det(J_n) == 0:  # Avoid singular Jacobian
            print("Jacobian is singular. No solution found.")
            return None

        delta = np.linalg.solve(J_n, -F_n) # Solve J_n * delta = -F_n for the
update step
        d_next = d_n + delta[0]
        D_next = D_n + delta[1]
        errors.append(np.linalg.norm(delta))

        if np.linalg.norm(delta) < tolerance:
            print(f"Converged to (d={d_next}, D={D_next}) after {i+1}
iterations.")
            return d_next, D_next, errors

        d_n, D_n = d_next, D_next

    print("Maximum iterations reached. No solution found.")
    return None
```

Lösningen i python är mycket lika lösningen från del 1.1 (Avsnitt 2.1), med skillnaden att derivatan är utbytt mot Jacobimatrisen, och att både d och D_s löses ut med hjälp av linjära algebra. Lösningen konvergerar till $\boldsymbol{d = 0.0118869m}$ och $\boldsymbol{D_s = 0.684476m}$ efter 9 itterationer.

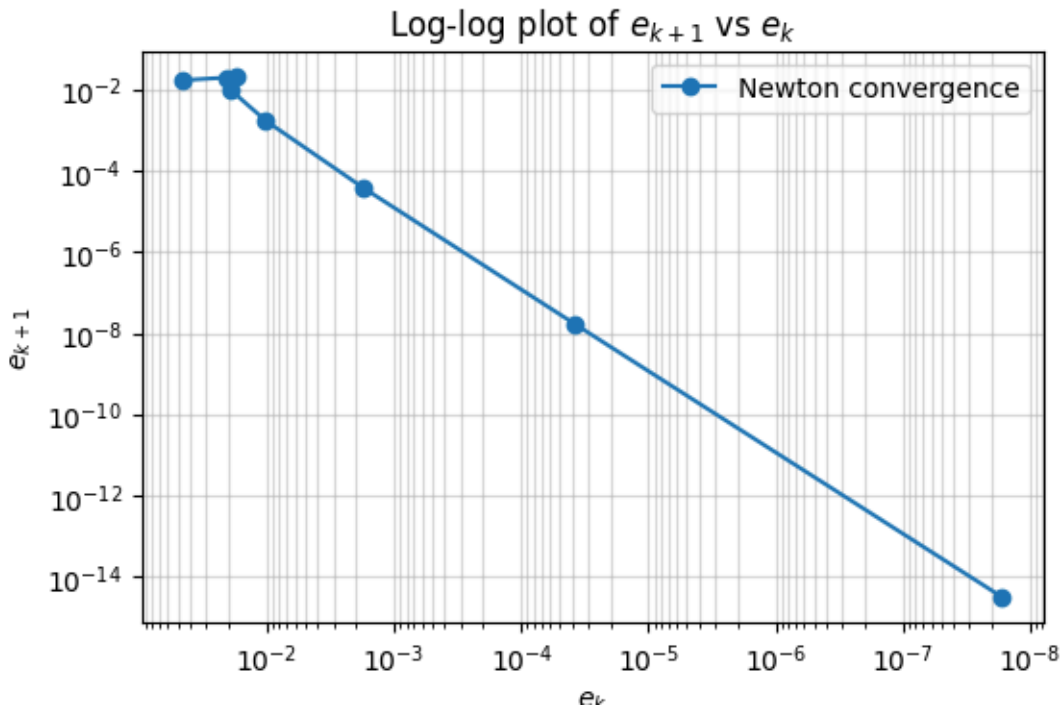
```
Initial guess: d = 0.015000, D = 0.800000
Iteration 1: d = 0.014483, D = 0.753866, Error = 4.61e-02
Iteration 2: d = 0.013815, D = 0.736671, Error = 1.72e-02
Iteration 3: d = 0.013018, D = 0.715934, Error = 2.08e-02
Iteration 4: d = 0.012303, D = 0.696451, Error = 1.95e-02
Iteration 5: d = 0.011947, D = 0.686244, Error = 1.02e-02
Iteration 6: d = 0.011888, D = 0.684513, Error = 1.73e-03
Iteration 7: d = 0.011887, D = 0.684476, Error = 3.76e-05
Iteration 8: d = 0.011887, D = 0.684476, Error = 1.66e-08
Iteration 9: d = 0.011887, D = 0.684476, Error = 3.14e-15
```

b) Genom att använda beräkna felet till normen av uppdateringssteget δ, kan konvergensen analyseras på samma sätt som i del 1.1, och uppskatta konvergensordningen p med Ekvation 24

```
for i in range(2, len(e)):
    order_of_convergence = np.log(e[i]/e[i-1]) / np.log(e[i-1]/e[i-2])
```

```
Order of convergence at iteration 2: -0.18984893081596307
Order of convergence at iteration 3: -0.33340935241195935
Order of convergence at iteration 4: 10.356263334193248
Order of convergence at iteration 5: 2.745105076264547
Order of convergence at iteration 6: 2.1575607548371583
Order of convergence at iteration 7: 2.017792532754369
Order of convergence at iteration 8: 2.0036658242583423
```

Detta ger en konvergensordning som i början svänger kraftigt, men som sedan stabiliseras runt **2**, vilket är i linje med den teoretiska konvergensordningen för Newtons metod ().



Figur 1: Log log

3.3 Del 2.1

3.3.1 Uppgift 6

a) Denna uppgift skall härleda den givna approximationen i Ekvation 12 genom att använda Taylorytveckling av funktionen f runt punkten $x + h$, och ta fram det specifika uttrycket för felet i approximationen, som är av ordning h^2.

$$f(x_j) = \frac{-\omega_{j+2} + 4\omega_{j+1} - 3\omega_j}{2h} + \mathcal{O}(h^2) = \frac{-f(x_j + 2h) + 4f(x_j + h) - 3f(x_j)}{2h} \tag{36}$$

$$f(x_j + 2h) = f(x_j) + 2hf'(x_j) + \frac{(2h)^2}{2}f''(x_j) + \frac{(2h)^3}{6}f'''(\xi_1) \tag{37}$$

$$f(x_j + h) = f(x_j) + hf'(x_j) + \frac{h^2}{2}f''(x_j) + \frac{h^3}{6}f'''(\xi_2) \tag{38}$$

Genom att substituera dessa uttryck i approximationen

$$\frac{-f(x_j) - 2hf'(x_j) - 2h^2f''(x_j) - 8\frac{h^3}{6}f'''(\xi_1) + 4f(x_j) + 4hf'(x_j) + 2h^2f''(x_j) + 4\frac{h^3}{6}f'''(\xi_2) - 3f(x_j)}{2h} \tag{39}$$

$$= \frac{2hf'(x_j) + 4\frac{h^3}{6}f'''(\xi_2) - 8\frac{h^3}{6}f'''(\xi_1)}{2h} = f'(x_j) + \frac{h^2}{3}(f'''(\xi_2) - 4f'''(\xi_1)) \tag{40}$$

$$= f'(x_j) + \mathcal{O}(h^2) \tag{41}$$

Där $\xi_1, \xi_2 \in [x_j, x_j + 2h]$. Detta visar att felet i approximationen är av ordning $\boldsymbol{h^2}$.

b) Denna uppgift skall implementera tre olika finita differnsmetoder i *Python* för att approximera derivatan av $\mathbf{sin}(\boldsymbol{e^x})$ vid $x = 0.75$. Resultatet skall jämföras med den exakta derivatan, samt analysera plotta felet i varje metod för olika värden av $h = 2^{-}k$, där $k = 1, 2, 3, 4, 5, 6, 7, 8$. detta gav följande resultat:

```
Analytical derivative at x=0.75: -1.09967e+00
h = 5.00000e-01, eq8: -1.30089e+00, eq12: -3.42609e+00, eq13: -5.70715e-01
h = 2.50000e-01, eq8: -1.17237e+00, eq12: -1.15732e+00, eq13: -9.30383e-01
h = 1.25000e-01, eq8: -1.11918e+00, eq12: -1.07648e+00, eq13: -1.05550e+00
h = 6.25000e-02, eq8: -1.10463e+00, eq12: -1.09127e+00, eq13: -1.08889e+00
h = 3.12500e-02, eq8: -1.10091e+00, eq12: -1.09734e+00, eq13: -1.09705e+00
h = 1.56250e-02, eq8: -1.09998e+00, eq12: -1.09907e+00, eq13: -1.09903e+00
h = 7.81250e-03, eq8: -1.09975e+00, eq12: -1.09952e+00, eq13: -1.09951e+00
h = 3.90625e-03, eq8: -1.09969e+00, eq12: -1.09963e+00, eq13: -1.09963e+00
```

,

3.4 Del 2.2

3.4.1 Uppgift 7

Denna uppgift skall diskritisera intervallet $[0, L]$ i $N = 4$ delintervall, och approximera andra derivatan med central differens, För att sätta upp det resulterande systemet av icke-linjära ekvationer och dess jacobimatris.

Intervallet $[0, L]$ diskretiseras i $N = 4$ delintervall, vilket ger 5 punkter: $x_0 = 0$, $x_1 = \frac{L}{4}$, $x_2 = \frac{L}{2}$, $x_3 = 3\frac{L}{4}$, och $x_4 = L$. Med dirichlet randvillkor, där $T(0) = T_s$ och $T(L) = T_\infty$, är de okända temperaturerna vid de inre punkterna $T_1 = T(\frac{L}{4})$, $T_2 = T(\frac{L}{2})$, och $T_3 = T(3\frac{L}{4})$.

Från Ekvation 14 och Ekvation 16 kan uttrycket för varje inre punkt skrivas som:

$$F \tag{42}$$

4 Diskussion

Discuss your findings and their implications.

5 Slutsats

Summarize your main findings and conclusions.